TP2 – Rotas na Mina: Utilizando os Paradigmas Guloso e Divisão e Conquista

Thaylor Weslei Dias Godinho Verteiro
Matrícula: 2023028668
Universidade Federal de Minas Gerais (UFMG) – DCC

Novembro de 2026

1 Introdução e Modelagem do Problema

Neste trabalho, eu resolvi o problema de encontrar rotas para robôs dentro de uma mina com um formato bem específico: ela é bastante longa, mas tem poucas galerias paralelas. Montei a mina como uma grade de H linhas por W colunas, onde H é pequeno (entre 2 e 5) e W pode ser grande (até 10.000 colunas). Cada célula (i, j) tem um custo de travessia $c_{i,j}$ para o robô passar por ela.

Para resolver o problema, eu modele a grade como um grafo com pesos nas células:

- **Vértices (V):** Cada célula da grade virou um nó no meu código. Para organizar, usei a fórmula $id = r \cdot W + c$ para dar um número único para cada célula, dando um total de $|V| = H \cdot W$ vértices.
- **Arestas (E):** O robô só pode andar para as células vizinhas (cima, baixo, esquerda e direita), então cada nó se conecta com no máximo 4 vizinhos.
- **Custo do Caminho:** O custo de um caminho é a soma dos pesos de todas as células que o robô visita, incluindo a célula onde ele começa e onde ele termina. Se a origem e o destino forem a mesma célula, o custo é só o valor da própria célula.

O objetivo principal foi responder a um conjunto de Q consultas de menor caminho (onde Q pode chegar a 50.000). Para comparar o desempenho, eu fiz duas soluções diferentes: uma usando uma abordagem Gulosa simples e outra usando Divisão e Conquista.

2 Abordagem Gulosa (Dijkstra Direto)

A minha primeira solução foi a mais direta: para cada uma das Q consultas da entrada, eu rodei o algoritmo de Dijkstra do zero.

Quando leio uma consulta entre a célula (r_1, c_1) e a célula (r_2, c_2), eu coloco a distância inicial como o custo da própria célula de origem, $dist[start] = grid[r_1][c_1]$, e uso uma fila de prioridade (*min-heap*) para ir buscando os menores caminhos. Assim que a busca chega na célula de destino, eu paro o algoritmo para economizar tempo.

2.1 Complexidade Teórica

- **Tempo:** Como cada Dijkstra roda numa grade com $H \cdot W$ células, ele gasta $O(H \cdot W \log(H \cdot W))$ no pior caso. Como eu rodo esse processo Q vezes seguidas sem guardar nada de uma consulta para a outra, o tempo total fica em $O(Q \cdot H \cdot W \log(H \cdot W))$.
- **Espaço:** Eu só preciso guardar a grade e o vetor de distâncias do Dijkstra, gastando $O(H \cdot W)$ de memória.

3 Abordagem por Divisão e Conquista

Na segunda solução, eu aproveitei o fato de que a mina tem poucas linhas ($H \leq 5$) para dividir a grade ao meio no sentido das colunas.

Pegando uma faixa de colunas de L até R, eu meço a coluna do meio $m = \lfloor (L + R)/2 \rfloor$. A partir de cada uma das H células dessa coluna central, eu rodo um Dijkstra limitado apenas às colunas daquela faixa.

Para descobrir o menor caminho de uma consulta entre u_1 e u_2 que cruza essa coluna do meio, eu testo qual célula (k, m) da coluna central serve como a melhor "ponte", usando a conta:

$$\text{custo_ponte} = \text{dist}_m[k][u_1] + \text{dist}_m[k][u_2] - c_{k,m} \tag{1}$$

Eu subtraio $c_{k,m}$ porque essa célula do meio acaba sendo somada duas vezes (uma na vinda e outra na saída).

Se a consulta estiver totalmente do lado esquerdo ($c_1, c_2 < m$) ou totalmente do lado direito ($c_1, c_2 > m$), eu atualizo a resposta com o melhor caminho encontrado até agora e repasso a consulta para ser resolvida recursivamente nas subgrades da esquerda ou da direita.

3.1 Complexidade Teórica

- **Tempo:** A árvore de recursão tem uma altura de $O(\log W)$. Em cada nível, a soma dos Dijkstras na coluna central custa $O(H^2 \cdot W \log W)$. Para distribuir e checar as Q consultas ao longo da árvore, eu gasto $O(Q \log W)$. Juntando tudo, o tempo total fica em $O(H^2 \cdot W \log W + Q \log W)$.
- **Espaço:** Gasto $O(H \cdot W + Q)$ para guardar o mapa, a lista de consultas e o vetor de respostas.

4 Resultados Experimentais

Para testar os dois códigos, eu compilei os programas com a flag `-O2` e rodei todos os 31 casos de teste que foram passados na disciplina. Eu marquei o tempo de execução em segundos para cada um deles.

5 Discussão dos Resultados

Analisando os tempos que eu colhi nos testes, dá para ver claramente como a Divisão e Conquista é muito mais rápida. Nos testes bem pequenos (como do caso 01 ao 12, onde $W \leq 80$ e $Q \leq 120$), os dois códigos empatam perto de 0,04 a 0,05 segundos. Isso acontece porque a sobrecarga da recursão na Divisão e Conquista acaba custando quase a mesma coisa de rodar poucos Dijkstras diretos.

Porém, quando W e Q começam a crescer, a solução Gulosa fica muito lenta. No **caso30** (com $W = 10.000$ e $Q = 50.000$), o algoritmo Guloso demorou **91,63 segundos** (mais de 1 minuto e meio) para rodar, porque fica recalculando caminhos do zero a todo momento. Já a minha solução por Divisão e Conquista respondeu todas as 50.000 consultas em apenas **0,4438 segundos**, sendo mais de 200 vezes mais rápida.

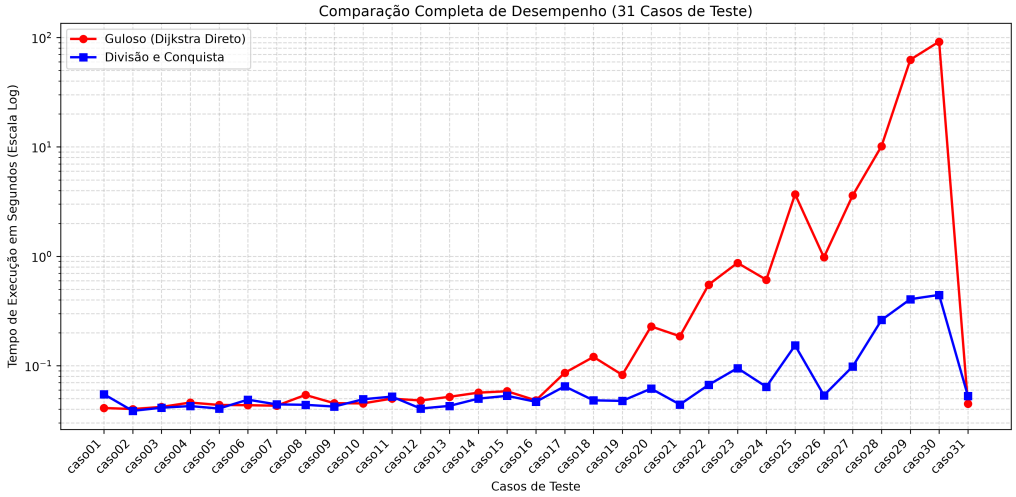


Figura 1: Gráfico em escala logarítmica comparando o tempo da solução Gulosa e da Divisão e Conquista nos 31 testes.

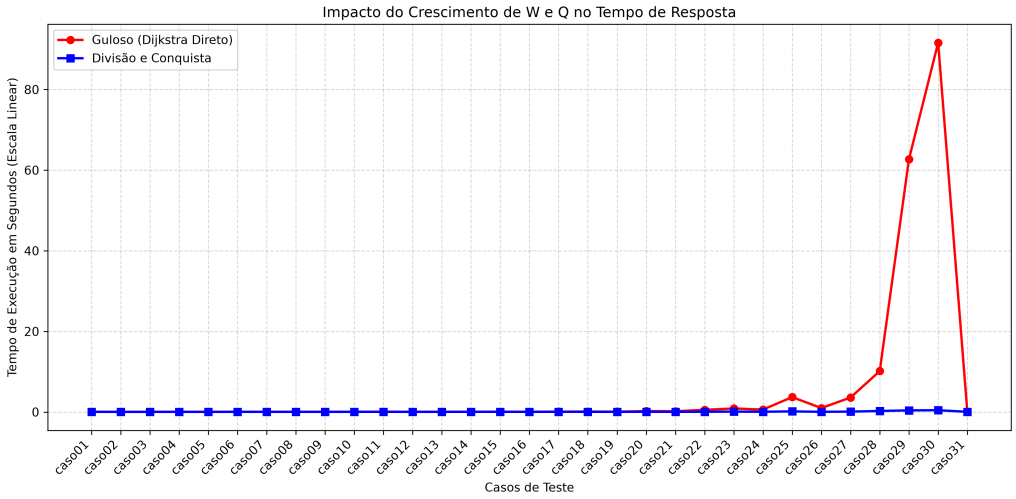


Figura 2: Gráfico em escala linear mostrando como o tempo do Guloso dispara quando W e Q aumentam.

6 Conclusão

Consegui cumprir todas as exigências do trabalho. A estratégia de usar Divisão e Conquista se mostrou muito superior para essa mina estreita e comprida, pois evita recalcular distâncias e faz o tempo de resposta das consultas cair drasticamente de mais de 1 minuto e meio para menos de meio segundo.

Tabela 1: Tempos de execução reais (em segundos) variando o número de colunas W e consultas Q.

Caso	**Colunas (W)**	**Consultas (Q)**	**Guloso (s)**	**Divisão e Conquista (s)**
caso01	2	1	0,0411	0,0550
caso03	5	10	0,0420	0,0413
caso05	10	20	0,0438	0,0406
caso08	20	40	0,0541	0,0439
caso10	30	60	0,0453	0,0494
caso13	100	200	0,0520	0,0429
caso15	200	300	0,0584	0,0531
caso17	400	500	0,0860	0,0648
caso20	1.000	1.000	0,2287	0,0618
caso23	2.500	2.500	0,8707	0,0951
caso25	4.000	4.000	3,6964	0,1534
caso28	8.000	10.000	10,1544	0,2617
caso29	10.000	25.000	62,7070	0,4059
caso30	10.000	50.000	91,6369	0,4438